

Project Report - Task 3

Data Storage Paradigms, IV1351

2024/11

Group 77 Project members:

[Sam Serbouti, serbouti@kth.se]

[Danni Noren, dannin@kth.se]

1 Introduction

This report details the work of creating an OLAP, Online Analytical Processing, queries and views along with an analysis of the efficiency of one of the queries with EXPLAIN ANALYZE of our imaginary music school : SoundgoodMusic using the relational database we previously built with PostgreSQL.

2 Literature Study

To complete this task, we have read chapters 6 and 7 of the 7th edition of *Fundamentals of Database Systems* by Ramez Elmasri and Shamkant Navathe that were about SQL and RDBMS standards. These chapters were fundamental to writing our queries.

We also followed a small online course (in French) on the topic of SQL by Nicolas Rangeon on the online learning platform openclassrooms, as well as an additional SQL tutorial on YouTube by Derek Banas, [link](#). When needed, we would go on [sqlTutorial.org](#) to obtain more documentation on functions and constructs (such as `TO_CHAR` for example).

This was in addition to the course's modules and lectures on relational models and SQL.

3 Method

We are using PostgreSQL as our DBMS and the pgAdmin tool to write and execute the SQL queries. To verify that the SQL queries work as intended we broke complex queries down to smaller simpler queries to ensure each step is working correctly, this

method simplified identifying the issues and debugging easier. For example, in case of nested queries we executed the inner query first and verified that the output is correctly commuted before writing the outer query.

4 Results

The git repository linking to the SQL queries can be found [here](#).

4.1 Number of lessons given per month during a specified year ANALYZED

It works by (left) joining the lesson table with the pricing_scheme table to associate each lesson with its pricing type. Then it filters the data to include only lessons that were marked as "given" and scheduled in 2024 and counts the total lessons and the number of lessons in each category (individual, group_lesson, and ensemble) using conditional CASE statements. It finally groups the data by the month of the lessons and sorts the output by the numeric month order.

Month	Total	Individual	Group	Ensemble
November	14	5	5	4

Table 1: Number of lessons given per month

See the complete analysis of this query in the discussion.

4.2 How many students there are with X siblings

The query doesn't rely on adding a column such as no_of_siblings to the student table. It dynamically calculates the number of siblings by joining the student table with the student_sibling table. This avoids the need to maintain a no_of_siblings column, which could be hard to keep up-to-date if a student quits, and ensures the solution remains normalized and flexible. Since the query uses a LEFT JOIN between student and student_sibling, it ensures that students with no siblings are still counted (students with no matching rows in student_sibling will have a sibling_count of 0).

The query seems appropriate for execution a few times per week, but if the data grows significantly, it may be worth considering optimizations. Indexes on student_sibling.student_id and student_sibling.sibling_id could help speed up the LEFT JOIN and the COUNT operations.

4.3 ids and names of all instructors who has given more than a specific number of lessons during the current month

For this, we joined three tables : instructor, person and lesson (the join conditions being on the instructor's id). We search for lessons that were given and extract their month and year to then filter so that it matches the current date. Finally, we group by instructor

number of siblings	number of students
0	8
1	8
2	3

Table 2: Number of students with X siblings

and filter out those that have given one or no lessons and order them by descending number of lessons.¹

instructor ID	First name	Last name	Number of lessons
1	William	Berg	4
2	Lucas	Larsson	4
3	Alicia	Lindberg	3
4	Filip	Sandberg	3

Table 3: Instructors who have given strictly more than 1 lessons in the current month

4.4 All ensembles held during the next week

This query obtains the day from the `scheduled_time` of the lesson, its genre and uses `COUNT(1.id)` to know the current number of students scheduled in it. A `CASE` construct allows us to categorize it as fully booked, empty or almost empty. We then join several tables : `lesson`, `group_lesson` and `ensemble` checking only for the ensembles which are scheduled 7 days from the current date. Lastly the data is grouped by the day, ensemble's genre and the number of the available seats and the output is sorted based on the day they are held and the genre of the ensemble.

day	genre	number of free seats
Fri	Experimental	Many seats
Sat	Jazz	Many seats
Sun	Orchestral	Many seats
Tue	Orchestral	Many seats
Mon	Pop	Many seats

Table 4: All ensemble held during the next week

¹this is optional, after a discussion with the customer, it might be better to order by instructor id

5 Discussion

In our generated data, both students and instructors are linked to the person table and share the same personal information. Although they have different roles within the database and do not cause any issues, we can update the data to ensure that students and instructors have different personal information.

5.1 Analysis of the first query

```
SELECT
    TO_CHAR(scheduled_time, 'Month') AS month_name,
    COUNT(*) AS total_lessons,
    COUNT(CASE WHEN ps.type_price = 'individual' THEN 1 END) AS individual_lessons,
    COUNT(CASE WHEN ps.type_price = 'group_lesson' THEN 1 END) AS group_lessons,
    COUNT(CASE WHEN ps.type_price = 'ensemble' THEN 1 END) AS ensemble_lessons
FROM
    lesson l
LEFT JOIN
    pricing_scheme ps ON l.price_id = ps.id
WHERE
    l.given = TRUE AND TO_CHAR(l.scheduled_time, 'YYYY') = '2024'
GROUP BY
    TO_CHAR(l.scheduled_time, 'Month'), TO_CHAR(l.scheduled_time, 'MM')
ORDER BY
    TO_CHAR(l.scheduled_time, 'MM');
```

We ran an EXPLAIN and ANALYZE on our first query to see how it performed. We used PostgreSQL's in-built analysis feature for this.

5.1.1 The statistics per node type

The different operations made during query execution are :

- aggregate = 1 (for counting the total number of lessons and counting the lessons specifically by type);
- index scan = 1 (on pricing_scheme to get the correct rows on the join condition between the lessons and their prices);
- nested loop left join = 1; seq scan = 1 (on the lesson table where given=TRUE and in 2024);
- Sort = 1 (sorted the data by month).

From this we can say that the index scan on the pricing_scheme is more efficient than a full table scan thanks to the index. The sorting can take a lot of resources but given our dataset size, it was reasonable to perform it. One aggregate and one sort also seem like a reasonable and acceptable number of functions

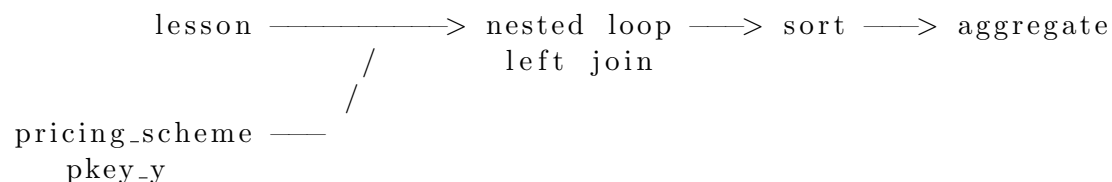
5.1.2 The statistics per relation

The tables involved in the query and that were accessed during execution are: `lesson` scanned once with a sequential scan (because there is no index on the `given` attribute or on the year); `pricing_scheme` scanned once using an index scan to get their ids quickly.

The tables that were accessed are those that we expected. The sequential scan on `lesson` means that PostgreSQL is scanning the entire table. We currently don't have a big dataset but in a music school, we can expect this table to be the largest one. If the customer is to use this query every week, this can become a problem. We could add a multi-column index on the columns `given` and `scheduled_time` like so :

```
CREATE INDEX idx_given_scheduled_time ON lesson (given, scheduled_time);
...
SELECT * FROM lesson
WHERE given = TRUE AND scheduled_time >= '2024-01-01';
```

5.1.3 Performance insight step by step



PostgreSQL reads rows sequentially from the `lesson` table, and filters them. For each filtered row in `lesson`, it finds the matching pricing scheme using the PK. It then combines both of them and arranges the results. Finally, it counts by group.

	Node	Rows/Actual	Loops
1	Aggregate	1	1
2	Sort	14	1
3	Nest Loop Left Join	14	1
4	Seq Scan on lesson as l	14	1
	<i>Filter (given AND (to_char(... etc)</i>	14	1
	<i>Rows removed by filter: 6</i>		
5	Index Scan using pricing_scheme_pkey	1	14
	<i>Index Cond: (id = l.price_id)</i>		

This once again shows that the `lesson` table was scanned sequentially (because of the lack of index on `given` and date) : loop = 1 so the scan was performed once. Also, it filtered 6 rows (about 30% didn't pass the WHERE condition).

5.1.4 Exploring views

If this query is to be performed a few times per week, it might be a good idea to create a view for it. In this case, we would change the query to add at the very beginning :

```
CREATE VIEW lesson_summary_2024 AS
... (same as before)
-- to use it:
SELECT * FROM lesson_summary_2024;
```

5.2 Analysis of the third query

```
SELECT i.id AS instructor_id, p.first_name AS "first name", p.last_name AS "last name",
       COUNT(l.id) AS "number of lessons"
FROM lesson l
JOIN instructor i ON l.instructor_id = i.id
JOIN person p ON i.person_id = p.id
WHERE l.given = true
       AND EXTRACT(YEAR FROM l.scheduled_time) = EXTRACT(YEAR FROM CURRENT_DATE)
       AND EXTRACT(MONTH FROM l.scheduled_time) = EXTRACT(MONTH FROM CURRENT_DATE)
GROUP BY i.id, p.first_name, p.last_name
HAVING COUNT(l.id) >= 2
ORDER BY "number of lessons" DESC;
```

5.2.1 The statistics per node type

The different operations made during query execution are :

- sequential scan = 1 (filters the given lesson during the current month and year),
- index scans = 2 (one collects the instructor's details and the other collects the instructor's first and last name),
- nested loop joins = 2 (first one joins the rows from lesson and instructor and the second one joins the rows from instructor and person),
- aggregation = 1 (counts how many lessons each instructor has provided in the current month and current year while filtering out the instructors with fewer than 2 lessons)
- sorting = 1 (sorts the instructors based on the number of lessons they have provided in a descending order).

5.2.2 The statistics per relation

As in the first query, the sequential scan on `lesson` in the `WHERE` clause can be slow in larger dataset which can be optimized by creating multicolumn index on the columns `given` and `scheduled_time` as seen below:

```
CREATE INDEX idx_lesson_scheduled_given ON lesson (given, scheduled_time);
```

In case of our dataset growing enabling hash or merge joins will improve performance as in nested loops every row of the table is compared with the row of the inner table.

```
SET enable_nestloop = OFF;
```

5.2.3 Performance insight step by step

	Node	Rows/Actual	Loops
1	sort	4	1
2	aggregate	4	1
3	sort	14	1
4	Nested Loop	14	1
5	Nested Loop	14	1
6	Seq Scan on lesson	14	1
	<i>Filter (given AND (EXTRACT(... etc)</i>	14	1
	<i>Rows removed by filter: 6</i>		
7	Seq Scan on instructor	2	14
8	Index Scan using person_pkey on person as p	1	14
	<i>Index Cond: (id = i.person_id)</i>		

PostgreSQL reads the `lesson` table sequentially and filters the rows (removing 6 rows), the filtered rows are then joined with the `instructor` table using the PK (19 rows were removed), in the second nested loop the `instructor` table is matched with `person` table using the PK using the `person` table, after joining the tables the query is grouped and the number of lessons are calculated while filtering the instructors with more than 2 lessons and finally the grouped results are sorted in a descending order. Performance summary can be seen in table below.

5.2.4 Exploring views

Since the query will run on a daily basis creating a materialized view can be beneficial where daily refresh is scheduled everyday or is done manually before usage.

```
CREATE MATERIALIZED VIEW instructor_lesson_summary_2024 AS
...(same as before)
-- to use it:
(REFRESH MATERIALIZED VIEW instructor_lesson_summary_2024;)
SELECT * FROM instructor_lesson_summary_2024;
```

5.3 Self assessment

Criteria	Notes
Are views and materialized views used in all queries that benefit from using them? Can any query be made easier to understand by storing part of it in a view? Can performance be improved by using a materialized view?	We have not used views and materialized views in every query however both were used in discussions for the first and the third query. Since all the queries will be used frequently, using them would be efficient. Performance can be significantly improved using materialized view specially in case of more complex queries as the third query in our case.
Did you change the database design to simplify these queries? If so, was the database design worsened in any way just to make it easier to write these particular queries?	One adjustment we needed to make in our database design was to add the column <code>instructor_id</code> to the <code>lesson</code> table in order to calculate the number of lessons each instructor has taught during a specific period.
Is there any correlated subquery, that is, a subquery using values from the outer query? Remember that correlated subqueries are slow since they are evaluated once for each row processed in the outer query.	We don't have any correlated subqueries in our queries as they are less efficient. We tried to avoid nested queries as much as possible.
Are there unnecessarily long and complicated queries? Are you for example using a UNION clause where it's not required?	We haven't used UNION in our queries. We tried to write the queries so they are easy to follow however in it possible that they can be written simpler more efficiently.

Table 5: Self-assessment based on the task's criteria

6 Comments About the Course

Using resources from the course, we were able to write our SQL queries. However we could have benefited from more text material on analyzing queries and optimize them.